

Building Production-Ready Agents

Core Execution: Runtime and Gateway

Amazon Bedrock AgentCore

Agenda

1. AgentCore Runtime: Serverless Execution
2. AgentCore Gateway: Tool Governance
3. Runtime and Gateway Architecture Flow
4. Key Security Boundaries
5. Key Takeaways

1. AgentCore Runtime

What is the Runtime?

The serverless execution engine hosting agent and tool code.

- **MicroVM Isolation:** Dedicated session compute limits cross-session access; tenant safety still requires correct identity and data mapping.
- **Lifecycle:** Runtime compute can be replaced at the configured maximum lifetime while the logical session continues.
- **MCP Support:** Native host for Model Context Protocol servers.

Amazon Bedrock API in the AgentCore Flow

Amazon Bedrock Runtime provides model inference; **AgentCore Runtime** hosts the agent code that makes those calls.

- `Converse` / `ConverseStream`: consistent message-based inference
- `InvokeModel` / `InvokeModelWithResponseStream`: model-specific inference
- AWS SDK clients use the `bedrock-runtime` service endpoint

Simplified request path: Runtime ↔ Bedrock model ↔ Gateway tools; Memory and Observability span the interaction.

`bedrock:InvokeModel` controls model access; AgentCore Gateway permissions control tool access.

Demo: Converse Under the Hood

REFERENCE ONLY

Run `day1/demos/demo-converse-under-the-hood/demo-converse-under-the-hood.ipynb` (follow-along notebook).

- Inspect real Bedrock responses and token usage
- Examine message history and tool-use exchange
- View the Converse API mechanics in action

Runtime in Production (Salesforce)

REAL-WORLD SCENARIO

Salesforce (Unified Planner):

- Salesforce built a single AI runtime (Unified Planner) that powers every Agentforce interaction across voice, chat, text, and MuleSoft. The runtime reduced response latency from ~20 seconds to 2.3 seconds by unifying previously separate execution architectures.
- Indeed deploys agents through API-driven workflows with immutable versioning: every agent is versioned like code, deployed through the same CI/CD pipelines as their services.

Runtime in Production (Retail and E-commerce)

SKIP**REAL-WORLD SCENARIO**

Retail and e-commerce (AWS shopping-agent pattern):

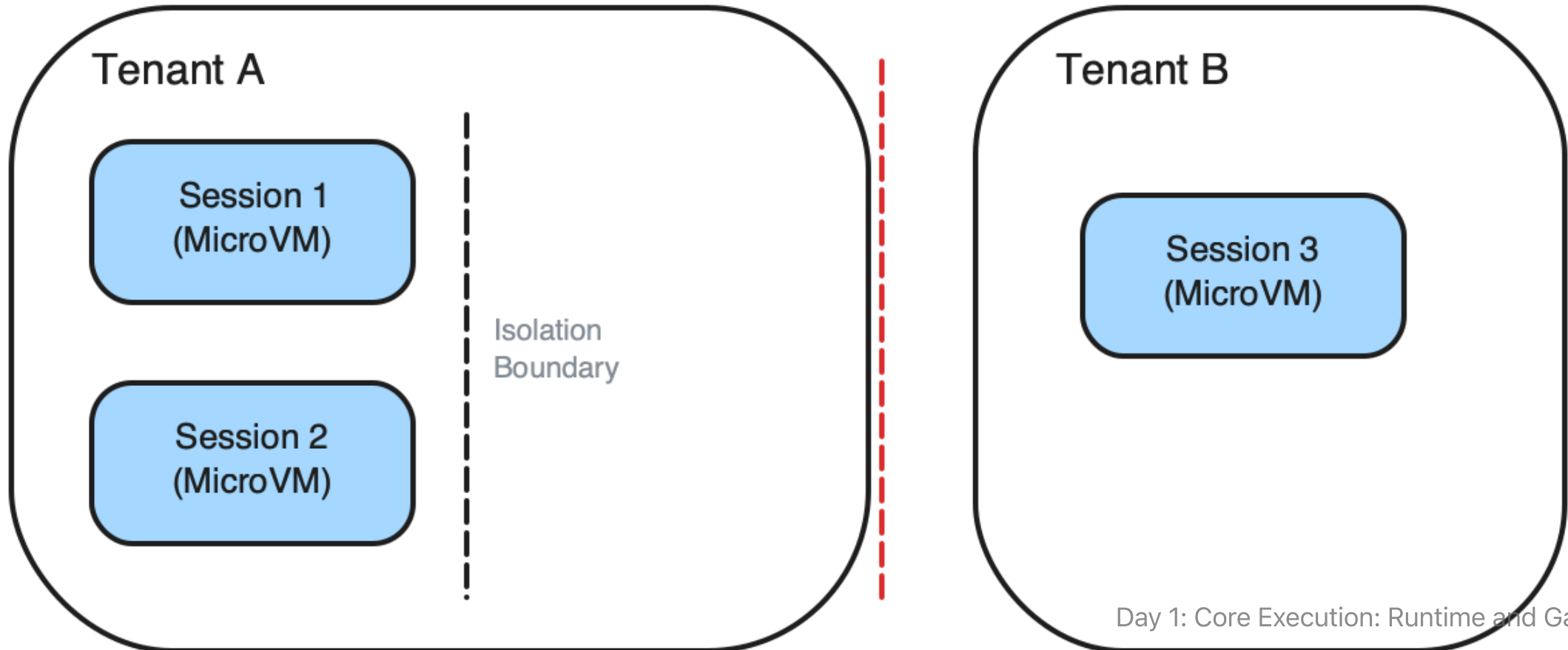
- AgentCore Runtime hosts a Strands shopping assistant that retrieves product candidates from Amazon OpenSearch Service and invokes approved commerce tools.
- Runtime isolates compute by session; the storefront backend must still map the authenticated shopper and tenant to the correct session and data filters.

Runtime Security Boundaries

- **Session vs. Tenant Isolation:** Session isolation does not authenticate the caller. The application backend must map authenticated users to session IDs.
- **Execution Role:** Code runs under an IAM (Identity and Access Management) execution role. Never use a broad development policy in production.

Runtime Security Boundaries

Runtime Security Boundaries



Session Isolation: Discussion

AgentCore Runtime gives every session its own microVM. Your application maps user IDs to session IDs in a Lambda function. One afternoon, a user reports seeing another user's data in their agent conversation.

DISCUSS

If microVMs provide hardware-level isolation, what went wrong?

Session Isolation: The Answer

MicroVM isolation protects what happens inside the session. It does not validate which user's data is loaded into it.

- **MicroVM isolation** prevents one session's memory from leaking into another session's microVM.
- **User-to-session mapping** is your application's responsibility. If the mapping is wrong, the wrong user's data enters the right microVM.
- **Defense:** Validate session ownership at the application backend before forwarding the session ID to Runtime.

Infrastructure isolation is necessary but not sufficient. Your application logic is the real gatekeeper.

2. AgentCore Gateway

What is the Gateway?

The centralized, governed entry point for agents to discover and invoke tools.

- **Protocol Translation:** Converts REST, Lambda, and Smithy models into standardized MCP tools.
- **Tool Governance:** Centralized point to allowlist tools, handle routing, and manage authorization.
- **Public Edge vs. Agent Edge:** Gateway is agent-facing. It does not replace Amazon API Gateway (which handles WAF, throttling, etc.).

Note: **WAF** stands for Web Application Firewall.

Governed Tool Access at Scale (Salesforce)

REAL-WORLD SCENARIO

Salesforce (Agentforce and governed actions):

- Salesforce actions expose approved CRM (Customer Relationship Management), Flow, Apex, MuleSoft, and external capabilities instead of granting a planner direct database access.
- In an AgentCore integration, Gateway provides the corresponding agent-facing tool boundary; Salesforce and downstream services still enforce business authorization.

Governed Tool Access at Scale

SKIP**REAL-WORLD SCENARIO**

Retail and e-commerce (AgentCore MCP pattern):

- A single Gateway can expose product search, order lookup, review submission, and returns while presenting a consistent MCP interface to the shopping agent.
- Keep order lookup read-only in class. Payment, refund, and order-placement tools need separate authorization, idempotency, limits, and approval controls.

Governed Tool Access at Scale (SaaS)

REAL-WORLD SCENARIO

SaaS (Salesforce — Indeed's API-Driven Agents):

- Indeed's agents access employer profiles, account status, and support history through a pre-processed Data 360 layer. Every tool call routes through governed APIs — agents never touch raw databases.
- Agent Script is authored in code and version-controlled. Tool allowlists are enforced at the Gateway layer, not in prompts.

Governed Tool Access at Scale (Healthcare)

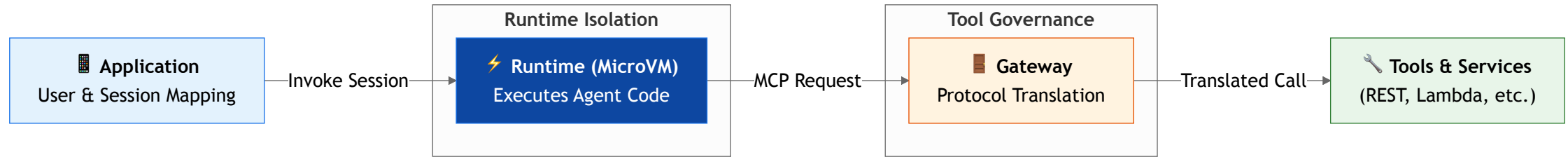
SKIP**REAL-WORLD SCENARIO**

Healthcare (UNC Health — Evidently Platform):

- UNC Health's Clinical Data Intelligence platform surfaces insights from the longitudinal patient record through governed API access. Clinicians query structured and unstructured EHR data through a single tool interface.
- The platform enforces HIPAA-compliant access controls at the Gateway layer — a clinician querying cardiology records cannot accidentally access oncology data without explicit authorization.

3. Architecture Flow

Runtime and Gateway Flow



4. Key Security Boundaries

API Security and Governance

- **Authorization:** A valid tool call via Gateway does not prove authorization. Downstream services must re-verify.
- **Unexpected Code Execution:** MicroVMs provide the first boundary, but prefer predefined tools and disable shell access.
- **Bypass Prevention:** Attach IAM conditions to prevent callers from bypassing the Gateway and directly invoking the Runtime endpoint.

Two Layers of Authorization

Your security team insists every tool call must be authorized at both the Gateway and the downstream service. An engineer argues this is redundant: "Gateway already validated the call."

- A) Right: Gateway authorization alone is sufficient; downstream checks add latency
- B) Right, but only because Gateway uses MCP protocol translation
- C) Wrong: Gateway only checks routing/allowlists; downstream must verify authz
- D) Wrong, but only for writes: read-only tools can skip downstream checks

Chat Check

Type your answer (A, B, C, or D) in the chat window now.

Two Layers of Authorization: The Answer

C: Gateway validates routing and allowlists; downstream services must independently verify business-level authorization.

The Gateway ensures the tool is on the allowlist and routes correctly. It does not know whether this caller should access this record. That's the downstream service's job. Defense in depth means both layers matter.

Key Takeaways

- **Decoupling:** Runtime provides compute; Gateway provides tool routing.
- **Scale:** Decoupling allows safe scaling without exposing backend tools to the execution environment.
- **Security:** Least-privilege IAM and explicit downstream authorization are mandatory for enterprise readiness.

Demo: Runtime Deployment

REFERENCE ONLY

Run `python3 day1/demos/demo-runtime-deploy.py`.

- Prints a validated `create_agent_runtime` request without creating resources
- Separates Runtime lifecycle settings from application-level loop limits
- Highlights cost controls: idle timeout, max lifetime, StopRuntimeSession

Predict the Bypass Control

An engineer with valid AWS credentials tries to invoke AgentCore Runtime directly, skipping Gateway entirely.

Which control actually stops that direct call?

- A) The Gateway's allowlist: it blocks callers that skip the translation step
- B) Amazon API Gateway's throttling rules: they apply to all AWS traffic
- C) The Runtime's resource policy: restricts calls to the approved backend principal
- D) IAM SigV4 (Signature Version 4) signing alone: any signed request is automatically authorized

Chat Check

Type your answer (A, B, C, or D) in the chat window now, with one reason.

Demo: Gateway Tool Access

Run `python3 day1/demos/demo-gateway-tool-access.py` .

- Prints a Gateway creation request without creating resources
- Executes a local MCP request-to-response transformation
- Distinguishes AgentCore Gateway from Amazon API Gateway
- Demonstrates direct bypass prevention with IAM conditions

Predict the Bypass Control: The Answer

C: the Runtime's resource policy, restricting invocation to the approved backend principal.

Gateway's allowlist (A) only applies once a request already reaches Gateway; it does nothing for a caller who skips Gateway and calls Runtime directly. Amazon API Gateway (B) isn't in this path at all; it fronts client traffic, not the agent-to-tool call. A valid SigV4 signature (D) proves who is calling, not whether they're allowed to. That's an authorization decision, not an authentication one. The actual boundary is Runtime's own resource policy: allow only the approved backend principal. Without it, a signed, allowlisted caller can still reach Runtime directly.

Questions?

Next: Kahoot Quiz 1, then a bio break